

CSE145 – Introduction to Computing

Assignment 2: Turing Machines — Design, Analysis & Simulation

This assignment is individually personalized — read the Personalization section before you start anything else.

Name: Ali Asad Sheikh ERP No.: 33412
Personalization block (fill in — see below): R = 12 V = 0 B = 1100 n = 6

Learning Objectives

- Design single-tape, deterministic Turing machines for both language recognition and function computation.
- Translate an informal problem description into a formal transition table / state diagram.
- Implement and test a Turing machine using an online TM simulator, and read a step-by-step execution trace.
- Diagnose a faulty machine by tracing its execution against its intended specification.

General Instructions

1. This assignment has 6 questions (Q1–Q6). Q1–Q5 each require four tasks: A (design), B (diagram/table), C (simulator implementation + trace evidence), D (design justification). Q6 has its own four tasks — see below.
2. Read the Personalization section FIRST and compute R, V, B, and n before attempting any question. Every question tells you which of these values it needs.
3. Submit a single PDF (preferred) or Word document containing, in order: the cover block above (filled in), then Q1 through Q6 in order, with all screenshots embedded directly in the document, plus the link to your GitHub repository (see "GitHub Repository for Simulator Evidence" below) stated clearly on the cover page.
4. Handwritten diagrams are acceptable for Task B if scanned clearly and inserted at the correct location; photos submitted as separate loose files will not be graded.
5. Deadline: Sunday, 7th Oct. 2026 at 11:55 PM. Late submission policy: 40% on missing the deadline, 20% on each subsequent day.

Personalization — Compute This First

Using the ERP number you are enrolled under, compute the following four values:

- R = the last two digits of your ERP number, as a number from 00 to 99.
- V = $R \bmod 4$. This is your variant number (0, 1, 2, or 3) — used to look up your specific parameters in the Master Variant Table below.
- B = the binary representation of R, written with no leading zeros (e.g. $R = 45 \rightarrow B = "101101"$; $R = 07 \rightarrow B = "111"$). This exact string is a required test input in Q1 and Q3, and forms part of the required input in Q5.
- n = (sum of ALL digits of your full ERP number, mod 5) + 3. This gives an integer between 3 and 7, used as a required length parameter in Q2 and Q4.

Worked example (NOT your answer — your ERP number will give different values): ERP number 24110 $\rightarrow$ digit sum = $2+4+1+1+0 = 8 \rightarrow 8 \bmod 5 = 3 \rightarrow n = 6$. $R = 10 \rightarrow V = 10 \bmod 4 = 2 \rightarrow B = "1010"$.

A submission with a missing or incorrect personalization block will not be graded, regardless of the quality of the machines inside it.

Approved Simulator

Implement every machine you design in AutomataVerse (www.automataverse.com/simulator) — a browser-based, visual simulator with Turing machine support. Do not use any other simulator, app, or self-written program for the required Task C evidence.

Single tape only. AutomataVerse supports multi-tape machines, but every machine in this assignment must be built and run as a SINGLE-TAPE Turing machine. A submission using more than one tape earns no credit for that question's Task C, regardless of correctness.

This assignment describes every machine using a generic 5-part transition notation: (current state, symbol read, symbol written, direction, next state). You must adapt this into AutomataVerse's own syntax — consult its built-in help or example machines for the exact format. Getting the syntax right is part of the exercise; a design that is correct on paper but never actually entered and run in the simulator earns no Task C credit.

GitHub Repository for Simulator Evidence

Create a single, private-then-shared-with-the-instructor GitHub repository for this assignment (name it clearly with your ERP number, e.g. 24110-cse145-a4-tm). For every Task C run required anywhere in this assignment, you must commit to that repository:

- the exported machine as a JSON file (AutomataVerse's JSON export), named so the question and run it belongs to are obvious (e.g. q1_run1_B.json);
- the AutomataVerse shareable link for that same machine, collected in a single README.md at the root of the repo, one line per run, labelled by question and run; and
- a screenshot of the halted run, placed in the same folder as its JSON file.

In your main submission document, for each Task C simply reference the corresponding file names / README line rather than re-pasting the JSON — but the repository itself is graded evidence, not optional backup, and must be shared with the instructor (via collaborator access or a link in your submission) before the deadline. A missing, empty, inaccessible, or post-deadline-modified repository is treated the same as missing Task C evidence.

How Each Question Will Be Verified — Read This Before You Start

Task C evidence. For every Task C, your GitHub repository must contain, for each required run: a full, uncropped screenshot of the finished, halted machine (tape, current/halting state, and transition diagram all visible); the exported JSON file for that exact machine and input; and the AutomataVerse shareable link, logged in your README.md. A screenshot or link with no matching JSON export, or a JSON export that doesn't open to the halted configuration shown in the screenshot, earns no Task C credit.

Personalized inputs. Several questions require you to run your machine on a value derived from your own ERP number (B or n above). The expected value for your specific ERP number will be independently recomputed during grading. Using someone else's personalized value, or a generic placeholder, earns zero for that task regardless of whether the machine itself is otherwise correct.

Design justification (Task D). Write your answer in your own words, referring to your own specific states and transitions. Generic or templated justifications that could apply to anyone's machine will be marked down; this section may also be checked with a text-similarity tool across the whole class's submissions.

Spot-check walkthroughs. A random subset of students will be asked, in a short in-person or video walkthrough, to explain any part of their own submission — a specific state, a specific trace, or why a transition is written the way it is. Being unable to explain your own submitted work in this walkthrough will be treated as a violation of the university's academic integrity policy, independently of whatever mark the written submission originally received.

Master Variant Table

Find your row for each question, then read across to the column matching your V. (Q6 has no variant — it is identical for everyone.)

Question	V = 0	V = 1	V = 2	V = 3
Q1 — Divisible-by-M Recognizer	$M = 3$	$M = 4$	$M = 5$	$M = 6$
Q2 — Scaled-Count Recognizer	$a^n b^n$	$a^n b^{2n}$	$a^{2n} b^n$	$a^n b^n c^n$
Q3 — Binary Transformer	+1 (increment)	−1 (decrement)	×2 (append a 0)	÷2 (drop last bit, floor)
Q4 — Palindrome-Family Recognizer	palindromes over {a,b}	palindromes over {a,b,c}	even-length-only palindromes over	anti-palindromes over {0,1}: string =

Question	V = 0	V = 1	V = 2	V = 3
			{a,b}	bitwise-NOT of its own reverse
Q5 — Marked-Comparison Recognizer	$\{ w\#w : w \in \{0,1\}^* \}$	$\{ w\#\text{reverse}(w) \}$	$\{ a\#b^i : i > j \}$	$\{ a^i\#b^j : i = j \}$

Personalization

3 3 4 12

$$R = 12$$

$$V = 12 \bmod 3 = 0$$

$$B = 1100$$

$$n = 3 + 3 + 4 + 1 + 2 = 13$$

$$13 \bmod 6 + 3$$

$$3 + 3 = 6$$

$$n = 6$$

Q1 — Divisible-by-M Recognizer (uses your V and your B)

Look up your value of M from the Master Variant Table using your V . Consider the alphabet $\Sigma = \{0,1\}$. Design a Turing machine that reads a binary string, most significant bit first, and accepts it if and only if the number it represents is evenly divisible by your M (0 counts as divisible by everything, and must be accepted).

Task A — In 4–6 sentences, describe in your own words the strategy your machine uses to track divisibility as it scans the string left to right. Think about what a single state needs to "remember" as the head moves — this is the same remainder-tracking idea as a DFA, but you must express it as an explicit Turing machine with a tape head that moves.

Task B — Provide the complete state diagram or transition table for your machine: all states, all transitions, and explicit accept and reject states.

Task C — Implement your machine in your chosen simulator. Run it on: (i) your personalized bit-string B ; (ii) one string of your own choice that IS divisible by M ; (iii) one string of your own choice that is NOT divisible by M . Commit the screenshot, JSON export, and shareable link for all three runs to your GitHub repository, and state in one line, for each run, whether the string was accepted or rejected and whether that is the correct answer for your M .

Task D — In 2–4 sentences, explain why a machine you built for one value of M could not be reused unmodified for a different value of M — what specifically would have to change?

Q2 — Scaled-Count Recognizer (uses your V and your n)

Look up your exact language from the Master Variant Table using your V . This is a language that a DFA cannot recognize — it requires unbounded counting — which is exactly why a Turing machine is needed here.

Task A — Describe your strategy in your own words. (A common approach is to repeatedly cross off one symbol from each "block" in a left-to-right-then-back sweep, checking the counts match the required ratio — but you may use a different valid strategy; describe whichever one you actually build.)

Task B — Provide the complete transition diagram/table.

Task C — Implement in your simulator. Run it on: (i) the string in your language built using exactly n = your personalized n repetitions of the base unit (e.g., for $a^n b^n$ with $n = 6$, that string is "aaaaaabbabbbb"); (ii) a string with one symbol added or removed so it falls just outside the language; (iii) one further string of your choice. Commit the screenshot, JSON export, and shareable link for all three runs to your GitHub repository.

Task D — Explain, in your own words, exactly where in your machine's design an off-by-one error would most likely occur, and how you convinced yourself your machine does not have one.

Q3 — Binary Transformer (uses your V and your B)

Look up your operation from the Master Variant Table using your V . Design a Turing machine that starts with a binary number written on the tape and halts with the tape containing only the binary result of applying your operation — no extra symbols, and any blanks left over from a shorter result must be cleaned up. This is a different kind of task from Q1/Q2: the machine computes an answer, it does not just accept or reject.

Task A — Describe your algorithm. (For $+1/-1$: think about how you would do grade-school carrying/borrowing, but moving across a tape rather than on paper, and which direction you must start from. For $\times 2/\div 2$: think about what a single shift does to a binary number, and which end of the tape you must shift from.)

Task B — Provide the complete transition diagram/table.

Task C — Implement in your simulator. Run it with your personalized bit-string B as the STARTING tape content. Commit a screenshot of the initial tape and a screenshot of the final halted tape (or one combined before/after screenshot), plus the JSON export and shareable link, to your GitHub repository. Underneath, write out by hand: the decimal value B represents, the decimal value your operation should produce, and confirm your machine's final tape content matches that value. (If your V selects decrement and B represents 0, first add 1 to R and recompute B before decrementing, and state clearly that you did this.)

Task D — Explain, in your own words, one edge case your machine must handle correctly that a naive "just modify the leftmost bit" attempt would get wrong.

Q4 — Palindrome-Family Recognizer (uses your V and your n)

Look up your exact language from the Master Variant Table using your V.

Task A — Describe the classic "mark the outer symbols and match inward" strategy as it applies to your specific variant. (For the anti-palindrome variant, describe how the comparison changes from checking equality to checking that the two symbols are complements of each other.)

Task B — Provide the complete transition diagram/table.

Task C — Implement in your simulator. Run it on: (i) a string of your own choosing, of length exactly $2n$ using your personalized n , that IS in your language; (ii) a string of length $2n$ that is NOT in your language; (iii) the empty string or a single symbol — whichever is the relevant boundary case for your variant. Commit the screenshot, JSON export, and shareable link for all three runs to your GitHub repository.

Task D — Explain, in your own words, why this strategy needs the tape head to make several full left-right sweeps rather than a single left-to-right pass, and what would go wrong with only one pass.

Q5 — Marked-Comparison Recognizer (uses your V and your B)

Look up your exact language from the Master Variant Table using your V. All four variants share one string containing a single delimiter symbol '#', with material to compare on either side of it.

Task A — Describe your strategy for using the tape head to repeatedly find and mark corresponding positions on either side of the '#'.

Task B — Provide the complete transition diagram/table.

Task C — Implement in your simulator. Construct a test string using your personalized B on the LEFT of the '#'. For the unary variants ($i > j / i = j$), first convert B to unary by writing one 'a' for every 1-bit and one 'b' for every 0-bit in B, and state clearly how you did this conversion. Choose the right-hand side so that the resulting string IS a member of your language in one run, and is NOT a member in a second run. Commit the screenshot, JSON export, and shareable link for both runs to your GitHub repository.

Task D — Explain, in your own words, why this language cannot be recognized in a single left-to-right pass, and what specifically about needing to "remember" the left-hand side forces the head to move back and forth.

Q6 — Debug the Broken Machine (identical for every student — no variant)

You are given the transition table below. It is INTENDED to accept binary strings over $\Sigma = \{0,1\}$ containing an EVEN number of 1s (the empty string has zero 1s, which is even, and must be accepted). The table contains exactly one incorrect entry. $\sqcup$ denotes the blank symbol.

Current State	Read	Write	Move	Next State
q_even (start)	0	0	R	q_even
q_even (start)	1	1	R	q_odd
q_even (start)	$\sqcup$	$\sqcup$	R	q_odd
q_odd	0	0	R	q_odd
q_odd	1	1	R	q_even
q_odd	$\sqcup$	$\sqcup$	R	q_reject
q_accept	(halting, accepting state — reached only via a correct transition)			
q_reject	(halting, rejecting state)			

Task A — By hand, WITHOUT a simulator, trace this exact machine on the input "" (the empty string) and on the input "11". Show the full sequence of configurations — state, full tape content, and head position — for each input, step by step, until the machine halts.

Task B — Based on your two hand traces, identify precisely which single row of the table above is wrong, and explain — using what your traces showed — why it is wrong and what the correct entry should be. Do not simply assert the fix; show how the trace revealed the problem.

Task C — Implement BOTH the broken machine exactly as given, and your corrected machine, in your simulator. Run the broken version on "", "11", "1", and "111"; run the corrected version on the same four strings. Commit the screenshot, JSON export, and shareable link for all eight runs to your GitHub repository, and provide a one-line summary table comparing broken vs. fixed vs. what the correct answer should be for each of the four strings.

Task D — In 3–5 sentences, reflect on which of your four test strings alone would NOT have revealed the bug, and what that tells you about choosing good test cases when checking your own future Turing machines.

Grading Weightage

Q1 – Q5 (each question weighted equally)

Task	Weight (per question)
A — Design description	15%
B — Diagram / transition table	25%
C — Simulator implementation + personalized evidence	40%
D — Design justification	20%

Q6

Task	Weight
A — Hand traces	20%
B — Bug identification	25%
C — Eight simulator runs	40%
D — Reflection	15%

$V=3$, $B=1100$

Q1 — Divisible-by-M Recognizer (uses your V and your B)

Look up your value of M from the Master Variant Table using your V. Consider the alphabet $\Sigma = \{0,1\}$. Design a Turing machine that reads a binary string, most significant bit first, and accepts it if and only if the number it represents is evenly divisible by your M (0 counts as divisible by everything, and must be accepted).

Task A — In 4–6 sentences, describe in your own words the strategy your machine uses to track divisibility as it scans the string left to right. Think about what a single state needs to "remember" as the head moves — this is the same remainder-tracking idea as a DFA, but you must express it as an explicit Turing machine with a tape head that moves.

Task B — Provide the complete state diagram or transition table for your machine: all states, all transitions, and explicit accept and reject states.

Task C — Implement your machine in your chosen simulator. Run it on: (i) your personalized bit-string B; (ii) one string of your own choice that IS divisible by M; (iii) one string of your own choice that is NOT divisible by M. Commit the screenshot, JSON export, and shareable link for all three runs to your GitHub repository, and state in one line, for each run, whether the string was accepted or rejected and whether that is the correct answer for your M.

Task D — In 2–4 sentences, explain why a machine you built for one value of M could not be reused unmodified for a different value of M — what specifically would have to change?

A) Divisibility in binary can be tracked by a method

similar to long division.
$$\begin{array}{r} 11 \\ 100 \overline{) 1100} \\ \underline{100} \end{array}$$

In my case $110 - 100 = 10$.

If a number is carried

$$\begin{array}{r} 0100 \\ \underline{100} \\ 0 \end{array}$$

forward, it is doubled and

b is added. $10 = 2$ in binary $2 \times 2 = 4 = 100$. $100 + 0 = 100$

we can write this expression as $(2r + b)$ and to

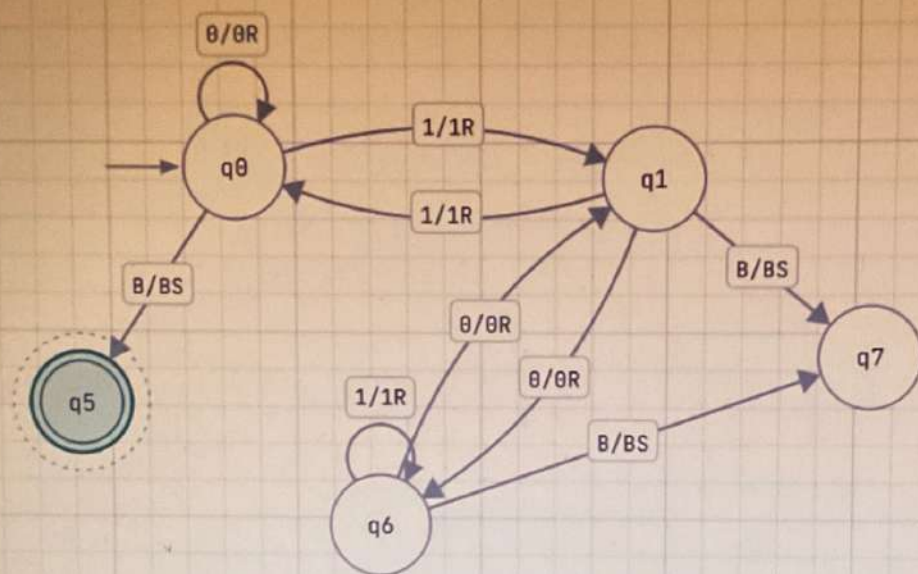
check divisibility, set it to mod 3. q_0 represents

a remainder at 0 q_1 , at 1 and q_2 at 2

B)

From	Read	move towards	To
q_0	0	right (R)	q_p
q_0	1	R	q_1
q_1	0	R	q_6
q_1	1	R	q_0
q_6	0	R	q_1
q_6	1	R	q_6
q_0	B	R	$(q_5) q_{\text{accept}}$
q_1	B	R	$q_{\text{reject}} (q_7)$
q_6	B	R	$q_{\text{reject}} (q_7)$





d) It will have a different mod and hence a
— different number of states for remainder

Q2 — Scaled-Count Recognizer (uses your V and your n)

Look up your exact language from the Master Variant Table using your V. This is a language that a DFA cannot recognize — it requires unbounded counting — which is exactly why a Turing machine is needed here.

Task A — Describe your strategy in your own words. (A common approach is to repeatedly cross off one symbol from each "block" in a left-to-right-then-back sweep, checking the counts match the required ratio — but you may use a different valid strategy; describe whichever one you actually build.)

Task B — Provide the complete transition diagram/table.

Task C — Implement in your simulator. Run it on: (i) the string in your language built using exactly n = your personalized n repetitions of the base unit (e.g., for $a^n b^n$ with $n = 6$, that string is "aaaaaabbbbb"); (ii) a string with one symbol added or removed so it falls just outside the language; (iii) one further string of your choice. Commit the screenshot, JSON export, and shareable link for all three runs to your GitHub repository.

Task D — Explain, in your own words, exactly where in your machine's design an off-by-one error would most likely occur, and how you convinced yourself your machine does not have one.

A) $A^n B^n$ means equal number of A and B.

I went with the simplest approach, I converted a to an x then traversed over to B and converted that to y.

Then I made the machine traverse left till it recognized x and then make a x again.

The approach given in the question is to

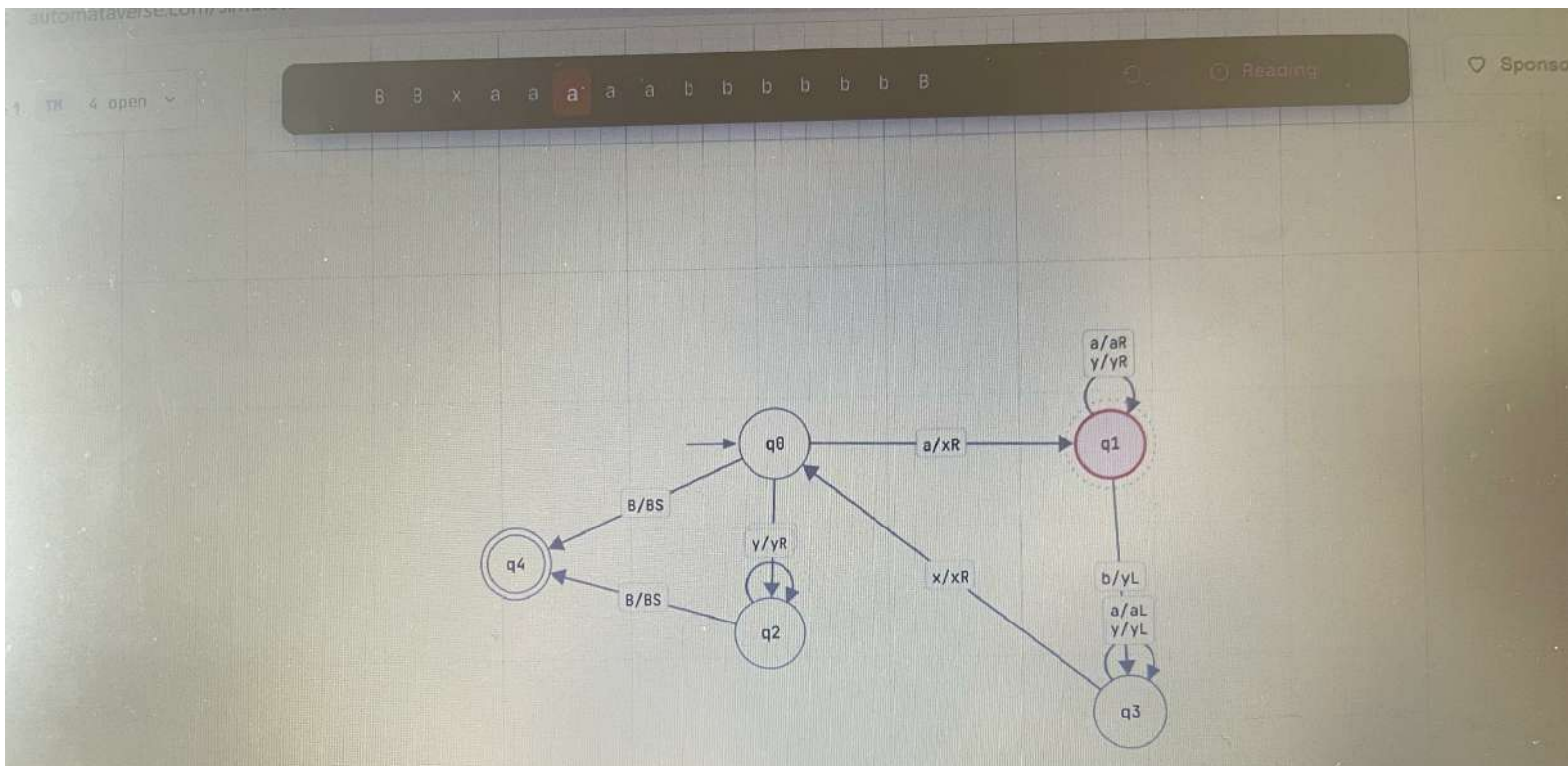
the approach given in the question. It has several self loops.

B)

From	Read	Write	Move	To
q_0	a	x	(right)R	q_1
q_0	y	y	R	q_2
q_0	B	B	stay (S)	q_4
q_1	a	a	R	q_1
q_1	y	y	R	q_1
q_1	b	y	left (L)	q_3
q_3	a	a	L	q_3

q_3	y	y	L	q_3
q_3	x	x	R	q_0
q_2	y	y	R	q_2
q_2	B	B	S	q_4

c)



d) When q_3 reads x/x_r and sends to q_0 , if it was one off it would re-mark an x or skip an a .
 I tested $5a's$ and $0b's$ and $60's$ and $5x's$.
 In the former it skips over the $x's$ and reaches B , in the latter all $0's$ become x , q_0 reads x and moves to q_2 which rejects b .

Q3 — Binary Transformer (uses your V and your B)

Look up your operation from the Master Variant Table using your V. Design a Turing machine that starts with a binary number written on the tape and halts with the tape containing only the binary result of applying your operation — no extra symbols, and any blanks left over from a shorter result must be cleaned up. This is a different kind of task from Q1/Q2: the machine computes an answer, it does not just accept or reject.

Task A — Describe your algorithm. (For $+1/-1$: think about how you would do grade-school carrying/borrowing, but moving across a tape rather than on paper, and which direction you must start from. For $\times 2/\div 2$: think about what a single shift does to a binary number, and which end of the tape you must shift from.)

Task B — Provide the complete transition diagram/table.

Task C — Implement in your simulator. Run it with your personalized bit-string B as the STARTING tape content. Commit a screenshot of the initial tape and a screenshot of the final halted tape (or one combined before/after screenshot), plus the JSON export and shareable link, to your GitHub repository. Underneath, write out by hand: the decimal value B represents, the decimal value your operation should produce, and confirm your machine's final tape content matches that value. (If your V selects decrement and B represents 0, first add 1 to R and recompute B before decrementing, and state clearly that you did this.)

Task D — Explain, in your own words, one edge case your machine must handle correctly that a naive "just modify the leftmost bit" attempt would get wrong.

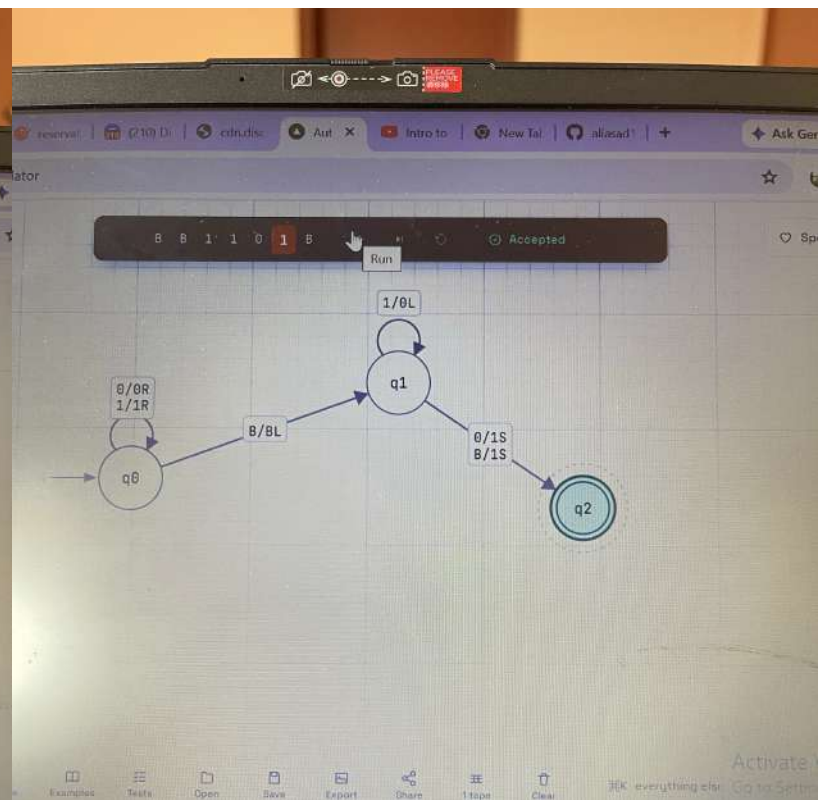
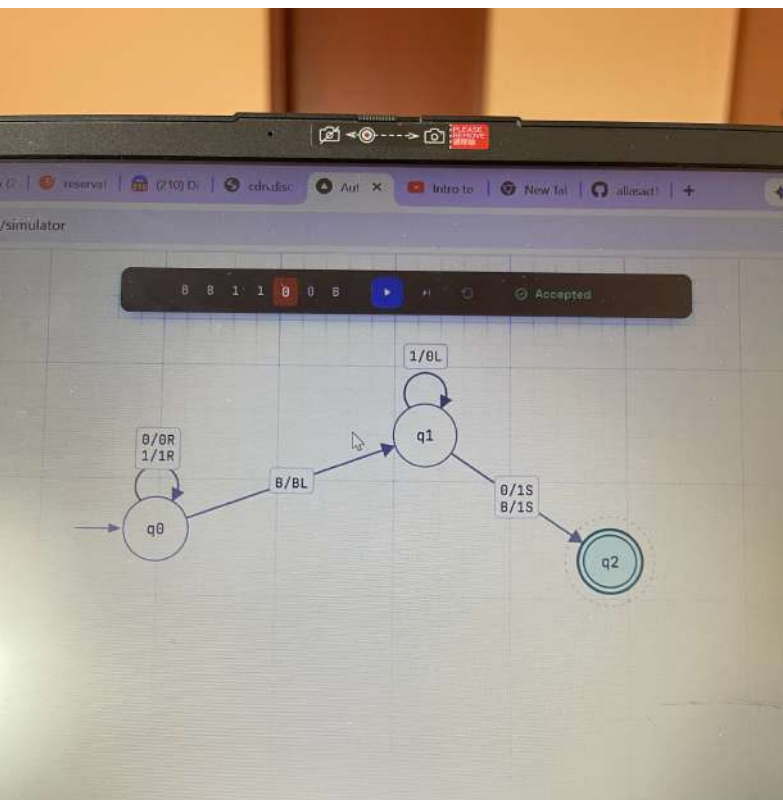
A) Firstly I use the in Built symbol to traverse to the right side. An incrementer adds 1 to the value. In binary, that basically means if the l.s.b is 0, move it 1 and if it is 1, traverse back to the zero you find and during the process make all 1's 0 and make that 0 into 1.

B) From Read Write Move To

q_0	0	0	R	q_0
q_0	1	1	R	q_0
q_0	B	B	L	q_1
q_1	1	0	L	q_1
q_1	0	1	S	q_2
q_1	B	1	S	q_2

Before

After.





↓

12

1000

↓

13

1101

✓

d) if all binary digits are 1, mine will make them all 0 and convert B to 1

For ex. 1111 $\rightarrow$ 10000. Only changing leftmost bit would skip that.

Q4 — Palindrome-Family Recognizer (uses your V and your n)

Look up your exact language from the Master Variant Table using your V.

Task A — Describe the classic "mark the outer symbols and match inward" strategy as it applies to your specific variant. (For the anti-palindrome variant, describe how the comparison changes from checking equality to checking that the two symbols are complements of each other.)

Task B — Provide the complete transition diagram/table.

Task C — Implement in your simulator. Run it on: (i) a string of your own choosing, of length exactly $2n$ using your

personalized n , that IS in your language; (ii) a string of length $2n$ that is NOT in your language; (iii) the empty string or a single symbol — whichever is the relevant boundary case for your variant. Commit the screenshot, JSON export, and shareable link for all three runs to your GitHub repository.

Task D — Explain, in your own words, why this strategy needs the tape head to make several full left-right sweeps rather than a single left-to-right pass, and what would go wrong with only one pass.

A) It's simple, you make a character on the leftmost into the initial B and you create two sub sections, one for a and the other for b. Then through self loop, you traverse to the right and compare that letter with the initial one, then you go to leftmost B and repeat.

	Read	Write	Move	To
from		B	R	q_1
q_0	a			
q_0	b	B	R	q_2
q_0	B	B	S	q_6
			R	q_1

q_1

q_1

q_1

q_2

q_2

q_2

q_3

q_3

q_4

q_4

q_6

q_6

q

b

B

q

$\downarrow$

B

q

B

b

B

q

b

q

$\downarrow$

B

q

b

B

B

B

B

B

q

b

R

L

R

R

L

L

$\downarrow$

L

$\downarrow$

L

L

q_1

q_3

q_8

q_8

q_4

q_6

q_6

q_5

q_6

q_6

q_5

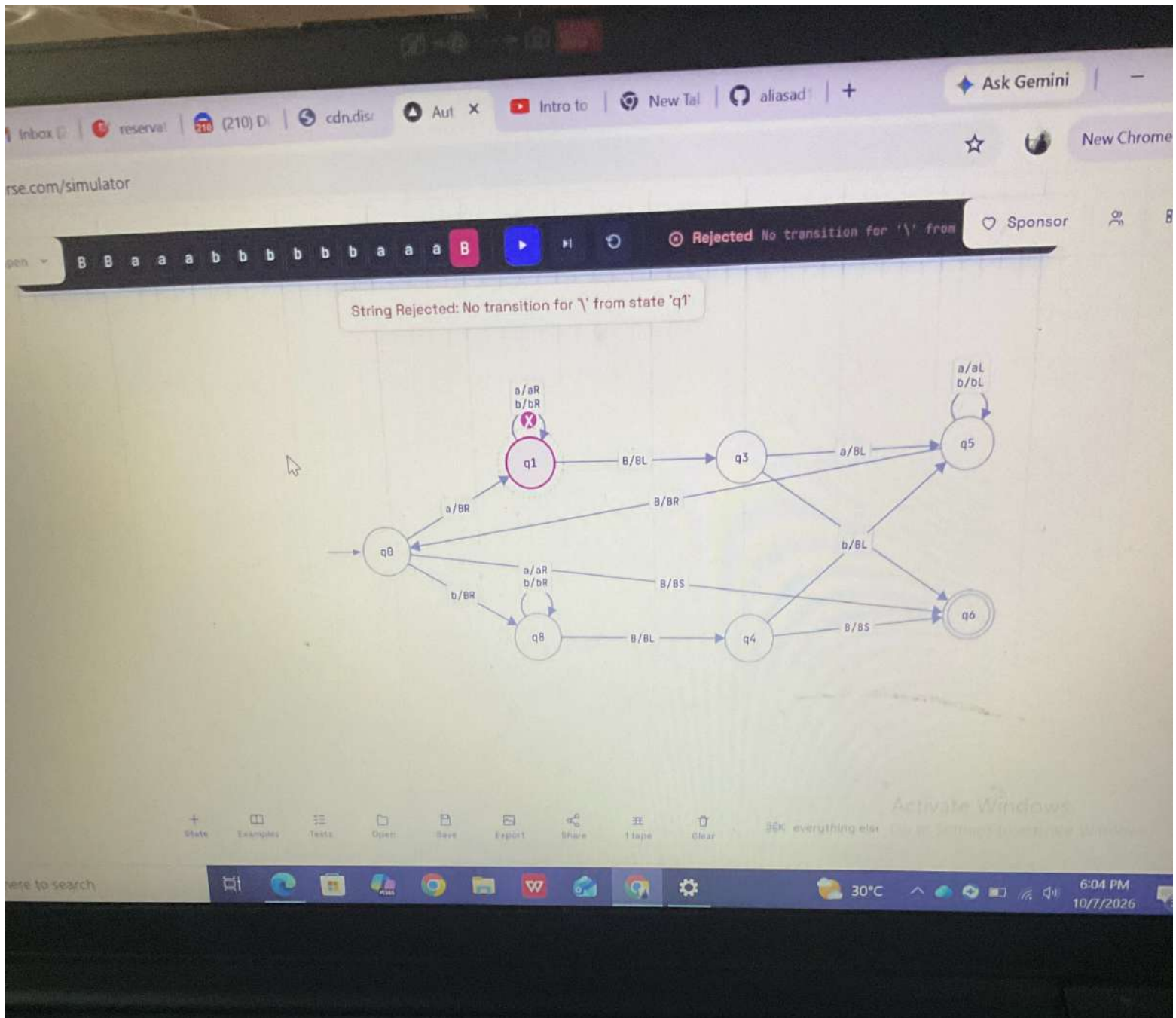
q5

B

B

R

q0



D) The alphabet on separate ends needs to be compared. Then the second last from left with the

from right and so on. Hence, many passes
needed. In one pass it will have passed
all the symbols before reaching the other
end. The head needs to travel back and forth.

Q5 — Marked-Comparison Recognizer (uses your V and your B)

Look up your exact language from the Master Variant Table using your V. All four variants share one string containing a single delimiter symbol '#', with material to compare on either side of it.

Task A — Describe your strategy for using the tape head to repeatedly find and mark corresponding positions on either side of the '#'.

Task B — Provide the complete transition diagram/table.

Task C — Implement in your simulator. Construct a test string using your personalized B on the LEFT of the '#'. For the unary variants ($i > j$ / $i = j$), first convert B to unary by writing one 'a' for every 1-bit and one 'b' for every 0-bit in B, and state clearly how you did this conversion. Choose the right-hand side so that the resulting string IS a member of your language in one run, and is NOT a member in a second run. Commit the screenshot, JSON export, and shareable link for both runs to your GitHub repository.

Task D — Explain, in your own words, why this language cannot be recognized in a single left-to-right pass, and what specifically about needing to "remember" the left-hand side forces the head to move back and forth.

A) Mark the leftmost symbol on the left half
that is unmarked, collect if it was 0,1.
Move past the # and x to first symbol on the
other side and compare it with the one on
the left. When left is fully marked, check
if right is fully marked as well

From	Read	write	moves downward	To
q_0	x	x	R	q_0
q_0	0	x	R	q_1
q_0	1	x	R	q_2
q_0	$\#$	$\#$	R	q_5
q_1	0	0	R	q_1
q_1	1	1	R	q_1
q_1	x	x	R	q_1
q_1	$\#$	$\#$	R	q_3
q_2	0	0	R	q_2
q_2	1	1	R	q_2
q_2	x	x	R	q_2

q_2	#	#	R	q_4
q_3	α	α	R	q_3
q_3	0	α	L	q_5
q_4	χ	χ	R	q_4
q_4	1	λ	L	q_6
q_6	0	0	L	q_6
q_6	1	1	L	q_6
q_6	χ	α	L	q_6
q_6	#	#	L	q_6
q_6	B	B	R	q_0
q_7	α	α	R	q_5

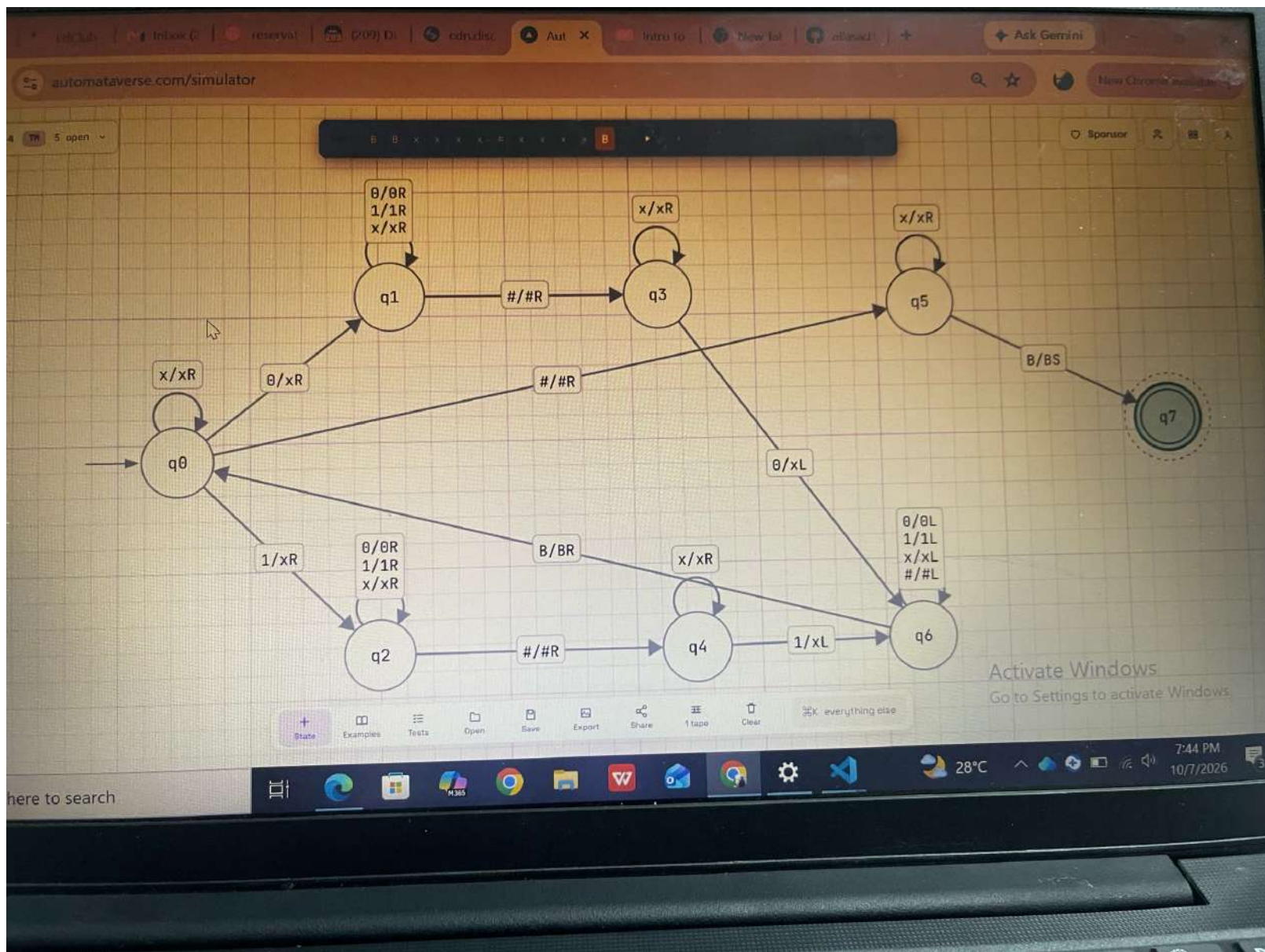
9₅

B

B

S

9₇



I was not able to completely understand
 what conver to a and b means and
 am short on time so im comparing

one that works and one that doesn't

D) It needs to compare digits from the left side with those from the right one by one as the machine has a finite number of states, while w can be as many as possible. One pass can not work because it has to store each state separately

Q6 — Debug the Broken Machine (identical for every student — no variant)

You are given the transition table below. It is INTENDED to accept binary strings over $\Sigma = \{0,1\}$ containing an EVEN number of 1s (the empty string has zero 1s, which is even, and must be accepted). The table contains exactly one incorrect entry. $\sqcup$ denotes the blank symbol.

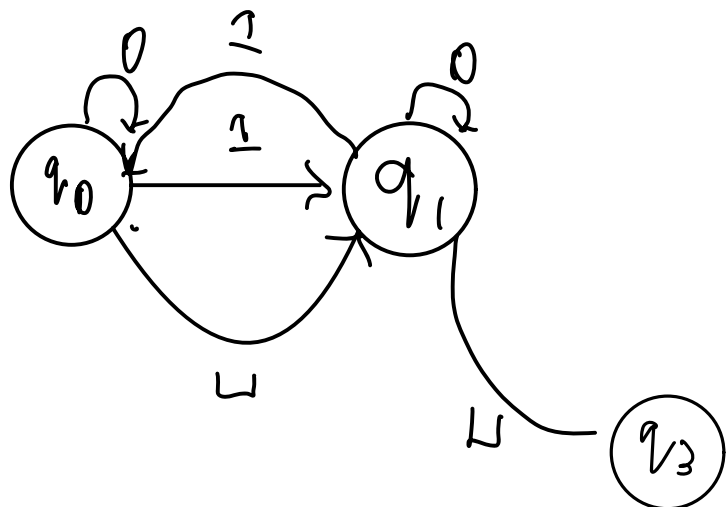
Current State	Read	Write	Move	Next State
q_{even} (start)	0	0	R	q_{even}
q_{even} (start)	1	1	R	q_{odd}
q_{even} (start)	$\sqcup$	$\sqcup$	R	q_{odd}
q_{odd}	0	0	R	q_{odd}
q_{odd}	1	1	R	q_{even}
q_{odd}	$\sqcup$	$\sqcup$	R	q_{reject}
q_{accept}	(halting, accepting state — reached only via a correct transition)			
q_{reject}	(halting, rejecting state)			

Task A — By hand, WITHOUT a simulator, trace this exact machine on the input "" (the empty string) and on the input "11". Show the full sequence of configurations — state, full tape content, and head position — for each input, step by step, until the machine halts.

Task B — Based on your two hand traces, identify precisely which single row of the table above is wrong, and explain — using what your traces showed — why it is wrong and what the correct entry should be. Do not simply assert the fix; show how the trace revealed the problem.

Task C — Implement BOTH the broken machine exactly as given, and your corrected machine, in your simulator. Run the broken version on "", "11", "1", and "111"; run the corrected version on the same four strings. Commit the screenshot, JSON export, and shareable link for all eight runs to your GitHub repository, and provide a one-line summary table comparing broken vs. fixed vs. what the correct answer should be for each of the four strings.

Task D — In 3–5 sentences, reflect on which of your four test strings alone would NOT have revealed the bug, and what that tells you about choosing good test cases when checking your own future Turing machines.



for ""

State	tape content	head	
q _{even}	␣	0	read ␣, write ␣ move R, go to q _{odd}
q _{odd}	␣␣	1	read ␣, write ␣, move R go to q _{reject}
q _{reject}	␣␣␣	2	halt and reject

for 11

State	tape content	head	
q_{even}	11	0	read 1, go to q_{odd}
q_{odd}	11	1	read 1, go to q_{even}
q_{even}	111	2	read 1, go to q_{odd}
q_{odd}	1111	3	read 1, go to q_{reject}
q_{reject}	11111	4	halt and reject

b) $q_{\text{even}} \rightarrow U, R$ q_{odd} is wrong,

11 and "" both are even strings, hence they
 should be accepted but they are not, in each
 case the first state is q_{even} . There is a
 contradiction $q_{\text{even}} \xrightarrow{1} q_{\text{odd}}$ and
 $q_{\text{even}} \xrightarrow{1} q_{\text{odd}}$. There is a problem of
 determinism. $q \xrightarrow{1} q$ is odd because 0+1 is odd
 but $q_{\text{odd}} \xrightarrow{1} q_{\text{odd}}$ should be q_{accept}

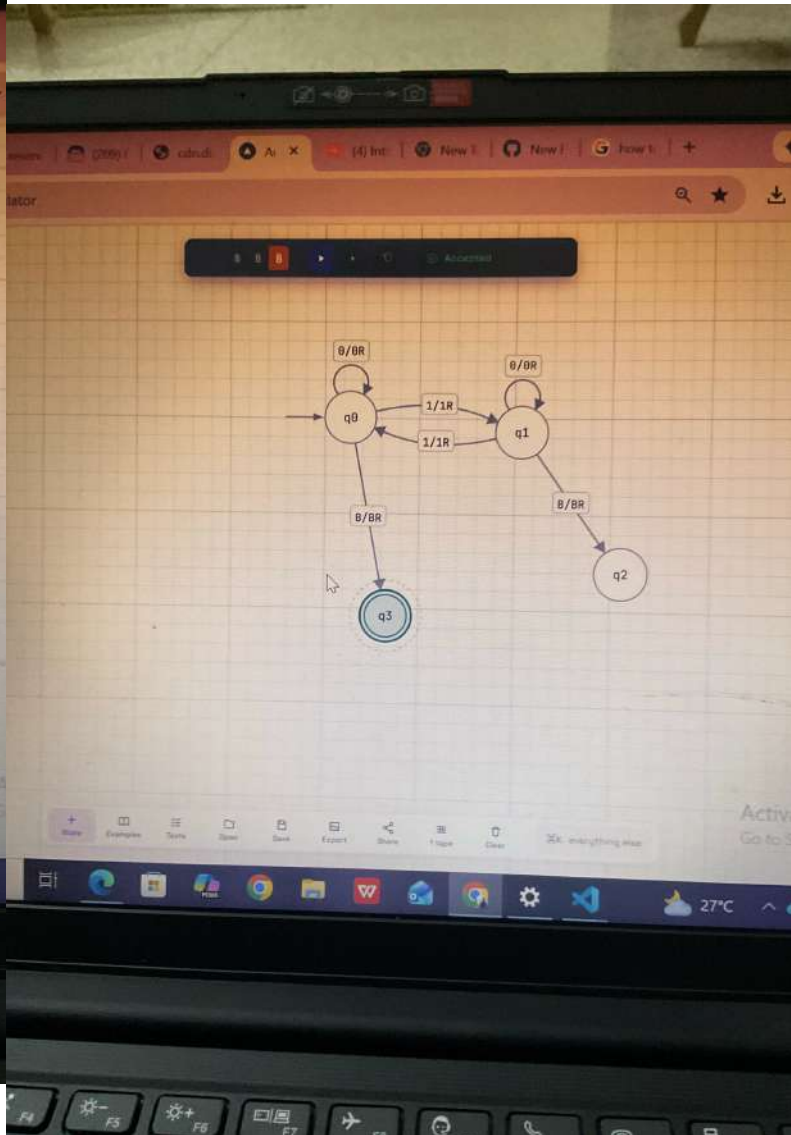
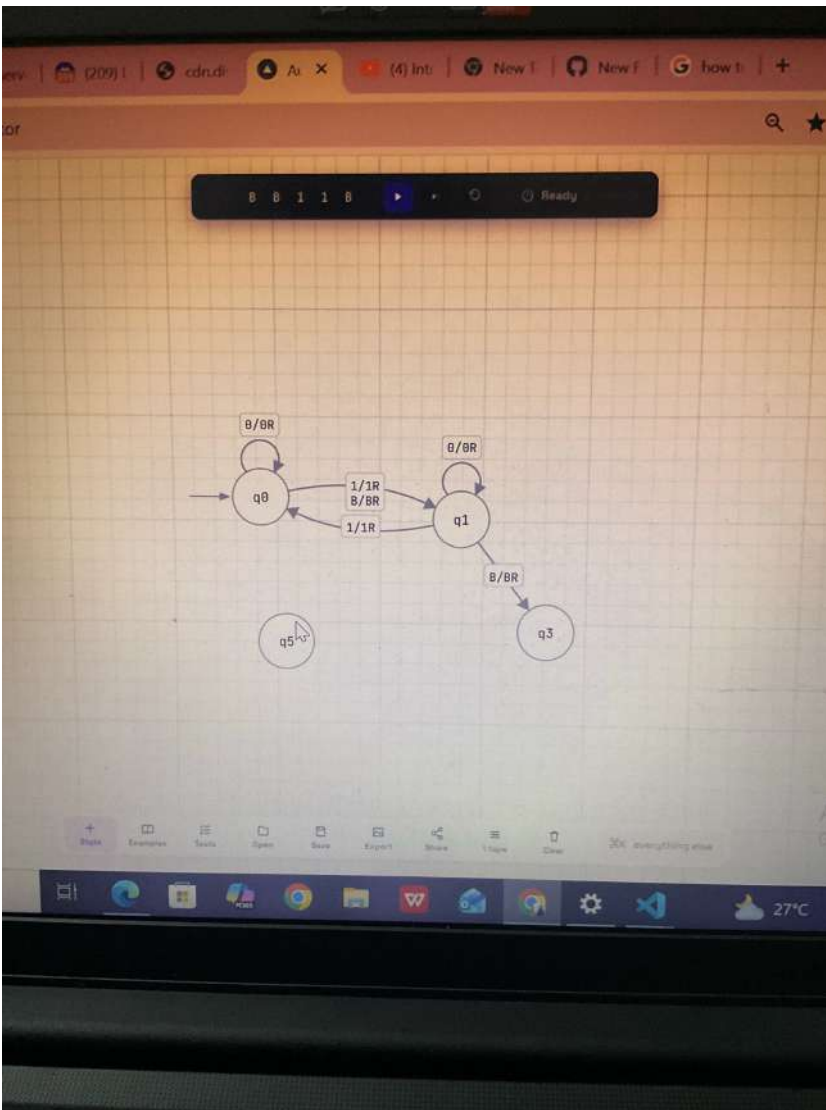
C)	Input	Broken	Fixed	Correct
	""	reject	accept	accept
	1	reject	reject	reject
	11	reject	accept	accept
	111	reject	reject	reject

Broken

Fixed

010011

11111



d) 1 and 111 would not have reached the

bug because they have odd no. of 1's
and reject is the correct answer but
" and 11 where machine should
accept but rejects identifies the bug.

You should choose test cases, that
accept and reject both.